

Secure Software Design and Development
(CY-321)

Content Security Problems?

Content Security Policy in the Wild

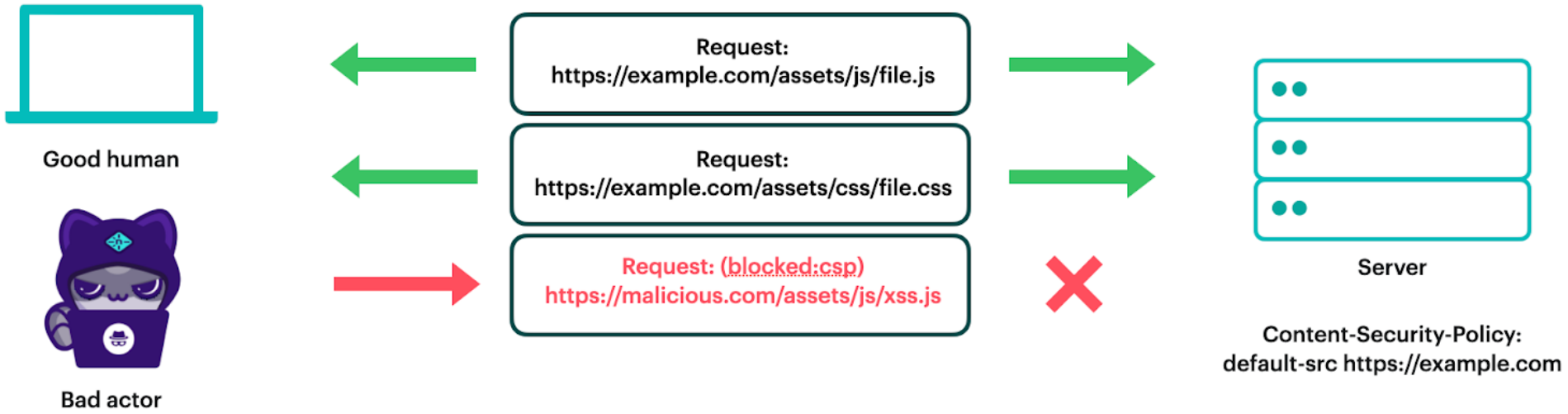
Content Security Policy

"a tool which developers can use to lock down their applications in various ways, mitigating the risk of content-injection vulnerabilities (. . .) and reducing the privilege with which their applications execute." [1]

[1] A. Barth, D. Veditz, and M. West. Content security policy level 2. W3C Working Draft, 2014

How to Tackle XSS

Content Security Policy



Content Security Policy



CSP is a W3C standard to protect web apps against content injection

Main Features

- a policy language for defining content restrictions
- legitimate content inclusion defined via white-listing
- inline scripts and styles are blocked by default
- dangerous functions like eval are banned by default
- two modes: enforcement and report-only (before enforcing)

How CSP works

The server sends a policy, usually as an HTTP header and the **browser** enforces it per page:

A policy is a list of **directives** separated by ;. Each directive names a content type and the **sources** allowed for it:

Content-Security-Policy: default-src 'self'; script-src 'self' https://cdn.example.com; img-src *

How CSP works

Directive	Controls
script-src	JavaScript
style-src	CSS
img-src	Images
connect-src	XHR, fetch, WebSocket targets
font-src	Fonts
default-src	Fallback for any type that has no directive of its own

How CSP works

Common source expressions are:

- **'self':** same origin (scheme, host and port)
- **https://cdn.example.com:** one specific host
- **https://*.example.com:** any subdomain
- **https:**: any HTTPS URL
- **'none':** nothing
- ***:** (almost) anything

Example

Policy at https://super-secure.com

```
script-src https://*.facebook.net;
default-src 'self';
```

Allowed Behaviours

- ✓ include scripts from https://connect.facebook.net
- ✓ include images, styles, etc. from https://super-secure.com

Blocked Behaviours

- ✗ include scripts from https://evil.com
- ✗ include images from http://super-secure.com
- ✗ run inline scripts

More on Inline Scripts

Inline scripts are dangerous, but pervasive and hard to remove!

CSP 1.0

Arbitrary inline scripts can be enabled using 'unsafe-inline':

```
script-src https://twitter.com 'unsafe-inline';
```

CSP Level 2

Selected inline scripts can be enabled using hashes and nonces:

```
script-src https://twitter.com 'nonce-a4jX67pq';
```

Research Goals

We want to assess the effectiveness of CSP in the current Web

Research Questions (RQs)

- do major web browsers implement CSP correctly?
- how large is the current CSP deployment?
- are existing policies accurate and secure?
- ~~how is the CSP deployment evolving over time?~~

[RQ1] Browser Support for CSP



Methodology

We designed a set of pages to test browsers against the CSP specification

- script-src http://a.com; script-src http://b.com
- script-src twitter.com
- ...

[RQ1] Enforcing Multiple Policies

The CSP specification mandates that when multiple policies are present on the same page all of them must be individually enforced

Example

- p1 = script-src http://a.com http://b.com
- p2 = script-src http://b.com
- ⇒ only http://b.com is a valid source for script inclusion

Still, Microsoft Edge allows script inclusion from http://a.com

[RQ1] Inconsistent Behaviours

Pick the following two policies:

- p1 = img-src http://a.com
- p2 = img-src http://a.com; default-src *

No 'unsafe-inline', but only p2 blocks inline scripts!

Security Analysis

- Unexpected behaviour according to the CSP specification
- Still, all the browsers we tested behave like this!
- “Luckily”, both p1 and p2 allow scripts from http://evil.com

[RQ2] Adoption of CSP

We crawled the Alexa Top 1M in March 2016, collecting CSP headers

- 8,133 websites deploy CSP (only 850 in March 2014)
- Shopify has a major impact (3,432 websites in report-only, 42.2%)
- only 3,220 websites actually enforce their policy

Provocative Thought

How many web developers are aware of their use of CSP?

[RQ2] Main Findings



Dangerous Constructs

- 51.8% of the websites activates 'unsafe-inline'
- only 2.6% of the websites adopts hashes or nonces
- 52.2% of the websites activates 'unsafe-eval'

Monitoring CSP

- only 21.6% of the websites monitors CSP violations
- 94 websites run CSP in report-only, but do not monitor violations!

Content Restriction?

- 21.6% of the websites use CSP just for frame busting

[RQ3] Configuration of CSP



Challenge

What does it mean to configure CSP correctly?

Classes of Errors

We defined and investigated four main classes of misconfigurations:

- typos: defalt-src 'self'
- ill-formed policies: script-src a.com b.com; c.com
- harsh policies: trigger CSP violations when accessing the website
- vulnerable policies: allow trivial script injection attacks (XSS)

[RQ3] Harsh Policies

Surprise! 17.2% of the websites trigger at least one CSP violation!

Type of Violation	#Violations / #Websites
Inline scripts	12 / 9
Inline styles	82 / 80
Use of eval	6 / 6
data: or blob:	43 / 33
HTTP(S) inclusion	778 / 441

75% of these websites do not monitor their policy...

[RQ3] Defining Weakness to XSS



Goal

Identify sufficient conditions on policies under which any content injection can be trivially turned into XSS by a web attacker.

Definition (Liberal White-List)

A white-list for script inclusion is liberal if and only if it contains any of:

- * ⇒ any host can provide scripts
- http: ⇒ any HTTP host can provide scripts
- https: ⇒ any HTTPS host can provide scripts
- data: ⇒ inline scripts can be included via data URIs

[RQ3] Weakness to XSS

Surprise! 92.4% of the websites fail against trivial script injections!

Reason for Vulnerability	#Websites
'unsafe-inline' in script-src	1,619
liberal white-list in script-src	259
no script-src + 'unsafe-inline' in default-src	275
no script-src + liberal white-list in default-src	175
no script-src + no default-src	1,024

How to Tackle XSS

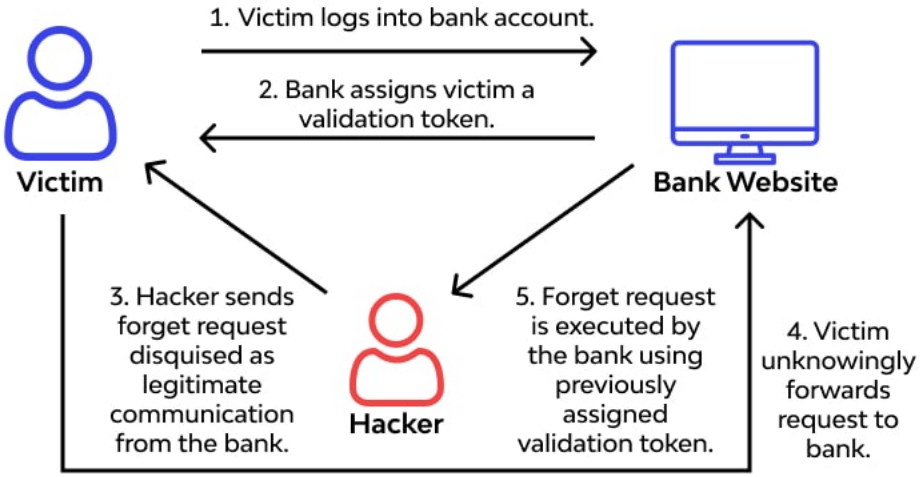
- Handle the output to the client only after it is sanitized
- Validating user supplied input with a whitelist
- Use the innerText properties of HTML controls instead of the innetHtml property when storing the input supplied

Rules to Tackle XSS

- Use secure libraries and encoding frameworks
- The client can be secured by disabling the active scripting option in the browser so that scripts are not automatically executed on the browser
- Use the HTTPOnly flag on the session or any custom cookie

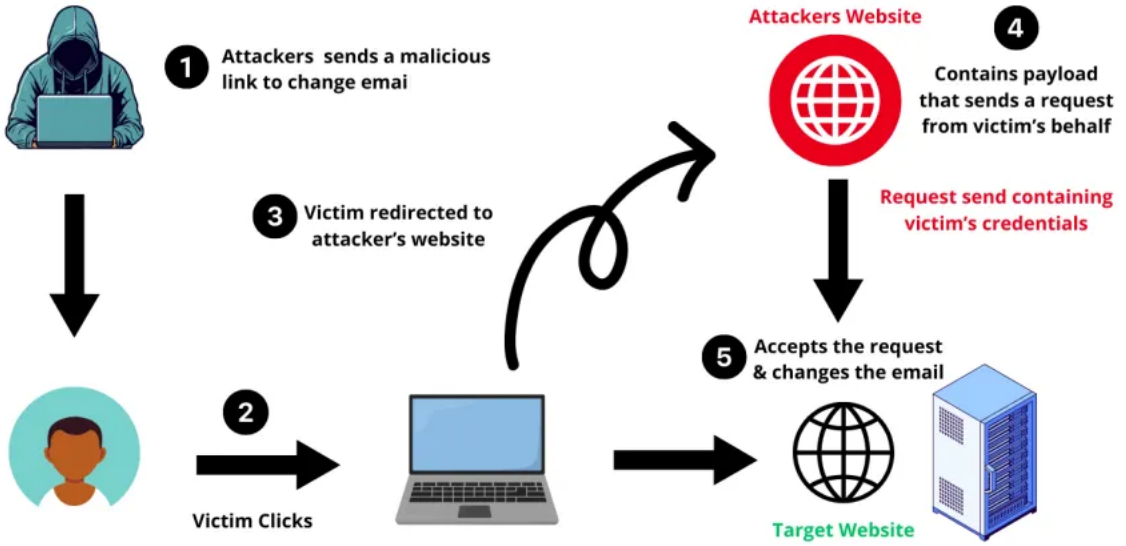
Cross-Site Request Forgery

- A user logs into their online banking account at bank.com.
- Their session remains active as they browse the internet.



- The victim visits the attacker's website while still logged into their bank account
- The malicious script automatically submits a request to bank.com, using the victims active session.

Cross-Site Request Forgery

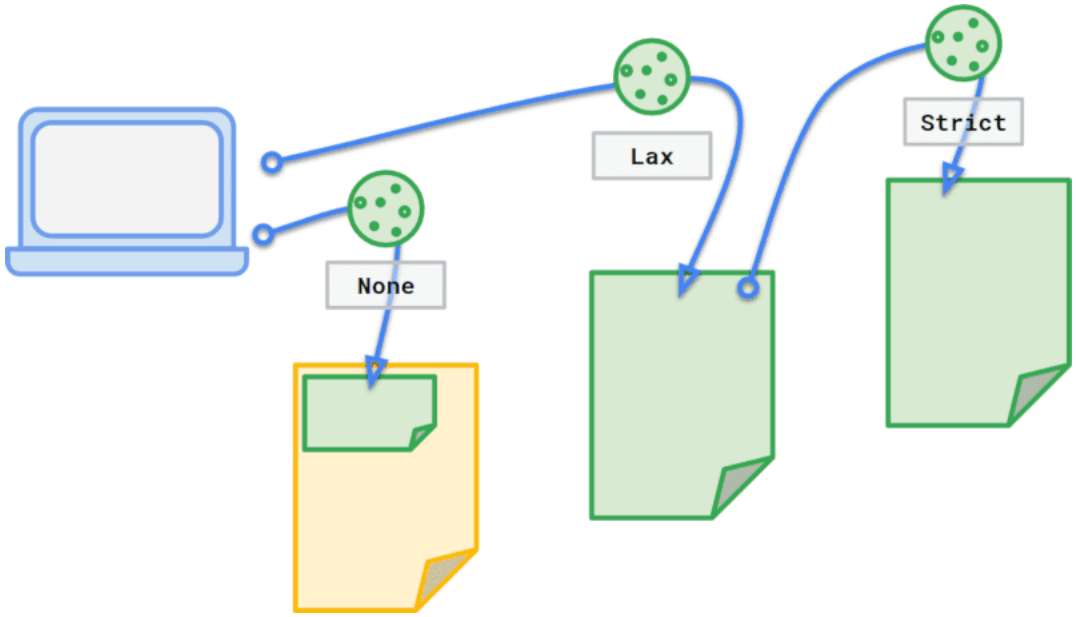


How to Tackle CSRF

- A CSRF token is a unique, unpredictable value included in requests to verify the request authenticity.
- The SameSite cookie attribute prevents browsers from sending cookies in cross-site requests.
- Force users to enter their password or use multi-factor authentication (MFA) before performing critical actions (e.g., money transfer, password change).
- Enable Content Security Policy (CSP)

How to Tackle CSRF

SameSite Cookie



SameSite cookie attributes (Strict``Lax``None) prevent Cross-Site Request Forgery (CSRF) by controlling if browsers send cookies with cross-site requests. Lax (default) allows cookies on top-level navigation (e.g., links)Strict limits them to the same domain, and None allows third-party context

Questions??

zubair.ahmad@giki.edu.pk

Office: G14 FCSE lobby